



 Latest updates: <https://dl.acm.org/doi/10.1145/3695991>

RESEARCH-ARTICLE

CodeScore: Evaluating Code Generation by Learning Code Execution

YIHONG DONG, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

JIAZHENG DING, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

XUE JIANG, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

GE LI, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

ZHUO LI, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

ZHI JIN, Key Lab of High Confidence Software Technologies, Ministry of Education, Beijing, China

Open Access Support provided by:

Key Lab of High Confidence Software Technologies, Ministry of Education



PDF Download
3695991.pdf
24 December 2025
Total Citations: 22
Total Downloads:
1871

Published: 23 February 2025

Online AM: 13 September 2024

Accepted: 19 August 2024

Revised: 16 July 2024

Received: 26 November 2023

[Citation in BibTeX format](#)

CodeScore: Evaluating Code Generation by Learning Code Execution

YIHONG DONG, JIAZHENG DING, XUE JIANG, GE LI, ZHUO LI, and ZHI JIN, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China

A proper code evaluation metric (CEM) profoundly impacts the evolution of code generation, which is an important research field in NLP and software engineering. Prevailing match-based CEMs (e.g., BLEU, Accuracy, and CodeBLEU) suffer from two significant drawbacks. 1. They primarily measure the surface differences between codes without considering their functional equivalence. However, functional equivalence is pivotal in evaluating the effectiveness of code generation, as different codes can perform identical operations. 2. They are predominantly designed for the Ref-only input format. However, code evaluation necessitates versatility in input formats. Aside from Ref-only, there are NL-only and Ref and NL formats, which existing match-based CEMs cannot effectively accommodate. In this article, we propose CodeScore, a large language model (LLM)-based CEM, which estimates the functional correctness of generated code on three input types. To acquire CodeScore, we present UniCE, a unified code generation learning framework, for LLMs to learn code execution (i.e., learning PassRatio and Executability of generated code) with unified input. Extensive experimental results on multiple code evaluation datasets demonstrate that CodeScore absolutely improves up to 58.87% correlation with functional correctness compared to other CEMs, achieves state-of-the-art performance, and effectively handles three input formats.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Code Evaluation, Code Pre-trained Language Model, Code Generation

This research is supported by the National Natural Science Foundation of China under Grant No. 62192733, 61832009, 62192731, 62192730, 62072007, the Key Program of Hubei under Grant JD2023008.

Authors' Contact Information: Yihong Dong, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: dongyh@stu.pku.edu.cn; Jiazheng Ding, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: dingjz@stu.pku.edu.cn; Xue Jiang, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: jiangxue@stu.pku.edu.cn; Ge Li (corresponding author), Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: lige@pku.edu.cn; Zhuo Li, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: lizhmq@pku.edu.cn; Zhi Jin, Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education; School of Computer Science, Peking University, Beijing, China; e-mail: zhijin@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/2-ART77

<https://doi.org/10.1145/3695991>

ACM Reference format:

Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. CodeScore: Evaluating Code Generation by Learning Code Execution. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 77 (February 2025), 22 pages.

<https://doi.org/10.1145/3695991>

1 Introduction

Automatic evaluation of code generation is significant and promising in the fields of **natural language (NL)** processing and software engineering. Due to the great potential of code generation in reducing development costs and revolutionizing programming modes, both industry and academia have devoted substantial attention to it [5, 9, 29, 35, 52, 60]. Code generation has achieved remarkable developments in the past few years [10, 14, 22, 27, 36], but **code evaluation metrics (CEMs)** still need to catch up. It is challenging to evaluate the competitiveness of various approaches without proper CEM, which hampers the development of advanced techniques for code generation. A range of code generation subtasks would benefit from valid code evaluation, including code completion [16, 32], code translation [50, 68], code search [1, 53], and so forth. Therefore, research on code evaluation is necessary and should be put on the agenda.

Some commonly used match-based CEMs treat code as text, such as BLEU [39] and Accuracy, which focus on basic and lexical-level features. They compute scores mainly based on n-gram co-occurrence statistics. CodeBLEU [48] additionally takes into account the structure of code, i.e., abstract syntax tree and data flow. However, the preceding CEMs have deficiencies in identifying code relationships, because code is mainly evaluated based on functional correctness rather than exact/fuzzy match to reference code, and match-based CEMs cannot account for the large and complex space of code functionally equivalent to reference code [20]. For example, in Figure 1, code (a) and code (b) have a much higher similarity of tokens or structures than code (c). However, through execution, we realize that code (a) and code (c) are different renderings of the same function. By contrast, the execution result of code (b) differs dramatically from both other codes, and code (b) even fails to compile. As a result, merely measuring the similarity of token/structure is insufficient for code evaluation.

Large language models (LLMs) pre-trained on code have demonstrated outstanding results in code generation tasks [5, 7, 8, 14, 29], which are fundamentally dependent on exceptional code comprehension. Excellent code comprehension is a crucial element for facilitating code evaluation. We hypothesize that LLMs pre-trained on code possess the ability to evaluate code. However, due to the training strategy of predicting the next token according to context, they lack awareness of evaluating code for functional correctness. Our objective is to instruct LLMs to evaluate code effectively in terms of functional correctness.

Another issue that requires resolution is that the existing match-based CEMs are exclusively confined to the Ref-only (consider only reference code) input format. This restriction presents three inherent disadvantages. First, for any code generation task, the correct solutions are not finite, but rather, they are inexhaustible. In this context, the provided reference code merely represents one correct solution among a vast multitude. Therefore, it is overly narrow to compare the generated code solely with one correct solution. Second, they neglect the NL description, which is a rich repository of information and a real requirement source. Third, these metrics are unusable in the absence of a reference code. This situation is quite commonplace in real-world evaluations where a correct solution is not always readily available. It is similar to code grading techniques in education, where grading often needs to be flexible and adaptable to different solutions that may not have a single correct answer. Therefore, expanding the input format of CEM is necessary.

<pre>def bubbleSort(arr): n = len(arr) for i in range(n): for j in range(0, n-i-1): if arr[j] > arr[j+1]: arr[j], arr[j+1] = arr[j+1], arr[j]</pre>	Reference Code (a) (a) <code>bubbleSort([5,3,2,1,4])</code> → [1,2,3,4,5]
<pre>def bubbleSort(arr): n = len(arr) for i in range(n): for j in range(0, n-i-1): if arr[j] == arr[j+1]: arr[j], arr[j+1] = arr[j+1], arr[j]</pre>	Generated Code (b) (b) <code>bubbleSort([5,3,2,1,4])</code> → <i>error</i> $BLEU(a, b) = 0.961$ $CodeBLEU(a, b) = 0.884$
<pre>def sortBubble (Nums): num_len = len(Nums) for j in range(num_len): sign = False for i in range(num_len - 1 - j): if Nums[i] > Nums[i+1]: Nums[i], Nums[i+1] = Nums[i+1], Nums[i] sign = True if not sign: break</pre>	Generated Code (c) (c) <code>sortBubble([5,3,2,1,4])</code> → [1,2,3,4,5] $BLEU(a, c) = 0.204$ $CodeBLEU(a, c) = 0.265$

Fig. 1. Results of evaluating the generated code implementing bubble sort using different CEMs. BLEU and CodeBLEU score the truly functional correct code (c) lower than the incorrect code (b).

In this article, we propose an effective LLM-based CEM, called CodeScore, which measures the functional correctness of generated codes on three input formats (Ref-only, NL-only, and Ref and NL). To obtain CodeScore, we present a code evaluation learning framework, UniCE, for tuning LLMs to estimate execution similarities with unified input. Specifically, we fine-tune LLMs to learn PassRatio and Executability of generated code, where Executability is devised to distinguish between compilation errors and output errors for code with PassRatio equal to 0. Generally, codes exhibiting higher functional correctness will pass more test cases, thereby achieving a higher PassRatio.¹ Consequently, for unexecutable codes, the model tends to assign scores approaching zero. In contrast, for codes demonstrating superior functional correctness, the model is likely to assign higher scores. CodeScore has the following advantages: (1) CodeScore has excellent evaluation performance, which achieves state-of-the-art performance correlation with functional correctness on multiple code evaluation datasets. (2) CodeScore provides three application scenarios (Ref-only, NL-only, and Ref and NL) for code evaluation with unified input, while traditional CEMs only consider Ref-only. Our major contributions can be summarized as follows:

- We propose an efficient and effective LLM-based CEM, CodeScore, that accommodates the functional correctness of generated codes from an execution viewpoint.²
- We present UniCE, a unified code evaluation learning framework based on LLMs with unified input, which assists models in learning code execution and predicting an estimate of execution PassRatio.³

¹Note that, although PassRatio varies across different test cases, it tends to yield a higher PassRatio for high-quality code, since we generate a large number of test cases. This phenomenon is somewhat akin to the process of human feedback. Despite the inherent variability in scores assigned by different human evaluators, the overarching trend remains consistent.

²<https://huggingface.co/dz1/CodeScore>

³<https://github.com/Dingjz/CodeScore>

- We construct three code evaluation datasets based on public benchmark datasets in code generation, called APPS-Eval, MBPP-Eval, and HE-Eval, respectively. Each task of them contains an NL description, several reference codes, 10+ generated codes, and 100+ test cases.⁴
- CodeScore substantially outperforms match-based CEMs and LLM-based **evaluation metrics (EMs)**, and achieves state-of-the-art performance on multiple code evaluation datasets.

2 Background and Related Work

In this section, we first introduce code generation, and then discuss code evaluation based on three types of EMs, including Match-based CEMs, Execution-based CEMs, and LLM-based EMs.

2.1 Code Generation

Code generation technology can automatically generate source code for software, achieving the purpose of machine-driven programming based on user requirements. Due to the rapid growth of code data and the continuous improvement of deep learning model capabilities, using deep learning for program generation has become the mainstream research direction [21, 31, 35, 43, 54, 58, 60, 65]. In recent years, the rise of pre-training techniques has provided new momentum for code generation. For example, studies like CodeT5 [57] and UniXcoder [15] pre-train models for completing code generation tasks. As the number of model parameters increases, researchers have observed the phenomenon of performance emergence in LLMs. LLMs such as AlphaCode [29], CodeGen [36], WizardCoder [33], ChatGPT [37], CodeGeeX [66], Starcoder [28], and CodeLlama [49] have demonstrated promising code generation performance. Currently, code generation technology and tools have been widely adopted in software development, such as Copilot [5], significantly enhancing the efficiency of developers. Assessing the quality of generated code has remained a critical problem in the development of code generation technology, directly influencing its advancement and evolution.

2.2 Code Evaluation

Match-Based CEMs. Besides these commonly used BLEU [39], Accuracy, and CodeBLEU [48], some niche CEMs [41] are also applied to code evaluation, e.g., METEOR [3], ROUGE [30], and CrystalBLEU [12]. However, these aforementioned match-based CEMs merely measure the surface-level differences in code and do not take into account the functional correctness of the generated code.

Execution-Based CEMs. They attempt to handle these issues by running tests for generated code to verify its functional correctness [17, 18, 25]. However, they come with several caveats: (1) It assumes that test cases have been given and all dependencies have been resolved. For each code generation task, supplying adequate test cases is a burden in practice, and the dependencies required vary from task to task. (2) Enormous computational overhead needs to be afforded. All generated code requires execution separately for each corresponding test case, which leads to enormous CPU and I/O overhead. (3) Execution with isolation mechanisms. The generated code could have some security risks, such as deleting files on the disk or implanting computer viruses, especially if the training data of code generation models is attacked. In a word, they are usually costly, slow, and insecure, which are often unavailable or ineffective in real-world scenarios.

LLM-Based EMs. Effective evaluation of generated results is hard for both text and code generation. They likewise face the same issue of poor EMs. A recent popular trend in evaluating text

⁴<https://github.com/YihongDong/CodeGenEvaluation>

generation is the design of automatic EMs based on LLMs. A part of LLM-based EMs [44, 45, 56] follows COMET [46] to learn high-quality human judgments of training data, which is a problem for code evaluation to obtain. Another part relies on LLM extracting token embeddings to calculate scores like BERTScore [63], such as [47, 51, 61, 64]. A concurrent work named CodeBERTScore [67] tries to use the same way as BERTScore with LLM pre-trained on code. However, they do not teach LLMs to learn code evaluation effectively, in other words, LLMs are still confused about how to evaluate code. Therefore, they exhibit suboptimal performance in code evaluation, as evidenced by our experimental results.

3 Methodology

In this section, we first introduce our proposed CEM CodeScore, and then describe a unified code evaluation learning framework (i.e., UniCE), which is used to yield the CodeScore.

3.1 CodeScore

For a code generation task $p \in P$, let the test case set of p as $C_p = \{(I_{p,c}, O_{p,c})\}_{c \in C_p}$, a set of paired test case input $I_{p,c}$ and test case output $O_{p,c}$. Although the potential program space can be boundless, test cases permit automatic evaluation of code generation capability. Thus, in contrast to most other text generation tasks, human judgment is not always necessary for code generation.

We measure the functional correctness with $PassRatio \in [0, 1]$, which is defined as

$$PassRatio = \frac{1}{|C_p|} \sum_{c \in C_p} \mathbb{I}\{\text{Eval}(g_p, I_{p,c}) = O_{p,c}\}, \quad (1)$$

where $|\cdot|$ indicates the element number of a set, $\mathbb{I}\{\cdot\}$ is an indicator function, which outputs 1 if the condition is true and 0 otherwise, and $\text{Eval}(g_p, I_{p,c})$ represents an evaluation function that obtains outputs of code g_p by way of executing it with $I_{p,c}$ as input.

Our framework UniCE can learn existing CEMs, including PassRatio and Passability.⁵ In this article, we choose PassRatio since we want to study execution similarity and continuous PassRatio can better reflect the execution similarity of different codes than binary Passability. In the case of generated code with PassRatio equal to 0, we also use binary *Executability* to distinguish whether the generated code can be executed successfully with all given test cases, and thus measure its quality

$$Executability = \begin{cases} 1, & \text{if code is executable,} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Given a unified input sequence $\mathbf{x}$ that admits the following three types, as shown in Figure 2:

- (1) *Ref-only* ($\mathbf{g} + \mathbf{r}$): Generated code concatenated with its reference code,
- (2) *NL-only* ($\mathbf{g} + \mathbf{n}$): Generated code concatenated with its NL description of requirements,
- (3) *Ref and NL* ($\mathbf{g} + \mathbf{r} + \mathbf{n}$): Generated code concatenated with both its reference code and NL.

UniCE yields a scalar CodeScore $\in [0, 1]$ and a binary number Exec

$$(\text{CodeScore}, \text{Exec}) = \text{UniCE}(\mathbf{x}), \quad (3)$$

where Exec = 1 if $\mathbf{g}$ can be executed successfully with all given test inputs otherwise 0, UniCE is our proposed learning framework, and details of UniCE are presented in Section 3.2.

⁵Passability is defined as $\frac{1}{|C_p|} \prod_{c \in C_p} \mathbb{I}\{\text{Eval}(g_p, I_{p,c}) = O_{p,c}\}$.

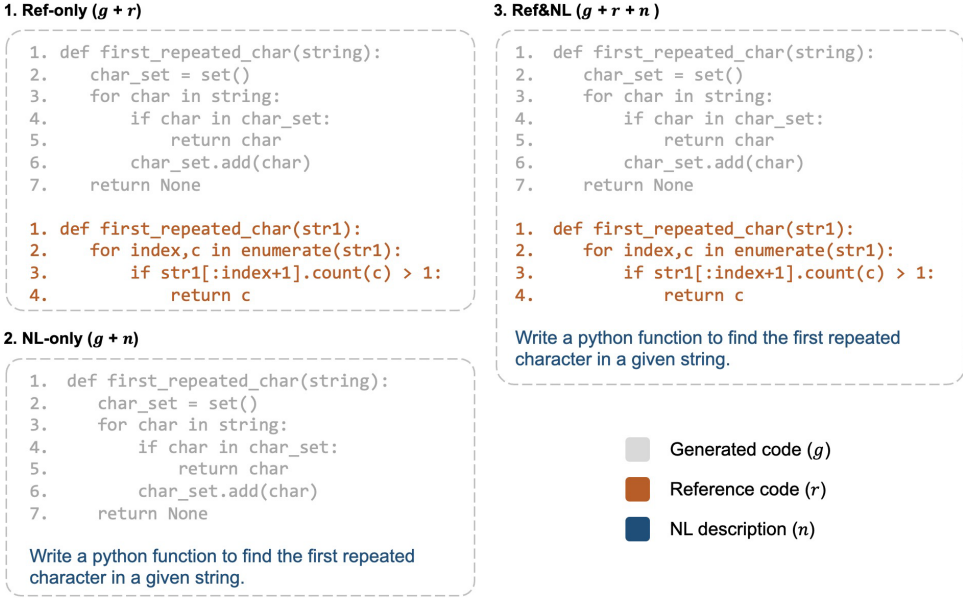


Fig. 2. Examples of three input formats for code evaluation.

We encourage UniCE to learn code execution (i.e., PassRatio and Executability) by minimizing loss function $\mathcal{L}$, which consists of two components

$$\mathcal{L} = \mathcal{L}_C + \mathcal{L}_E, \quad (4)$$

where $\mathcal{L}_C$ focuses on predicting PassRatio, and $\mathcal{L}_E$ on predicting code execution correctness. $\mathcal{L}_C$ and $\mathcal{L}_E$ are defined as

$$\mathcal{L}_C = (\text{CodeScore} - \text{PassRatio})^2, \quad (5)$$

$$\mathcal{L}_E = -\log p(\text{Exec} | \text{Executability}), \quad (6)$$

where $\mathcal{L}_C$ measures the squared difference between the predicted CodeScore and the actual PassRatio. $\mathcal{L}_E$ represents the negative log of the conditional probability of Exec given its Executability. The conditional probability is modeled as

$$p(\text{Exec} | \text{Executability}) = \begin{cases} p(\text{Exec}), & \text{if Executability} = 1, \\ 1 - p(\text{Exec}), & \text{otherwise,} \end{cases} \quad (7)$$

where $p(\text{Exec})$ is the predicted probability of successful execution.

3.2 UniCE

UniCE relies on LLMs to extract representations of $\mathbf{x}$ and can work with existing pre-trained LLMs. A detailed illustration of the UniCE framework is presented in Figure 3.

3.2.1 Pooling Layer. For LLMs, the pooling layer plays a critical role in enhancing the model's ability to capture and utilize information more effectively. The work [46, 55, 63] shows that exploiting information from different layers of LLM generally results in superior performance than only the last layer. Therefore, following the work [40], we pool information from different layers by using a

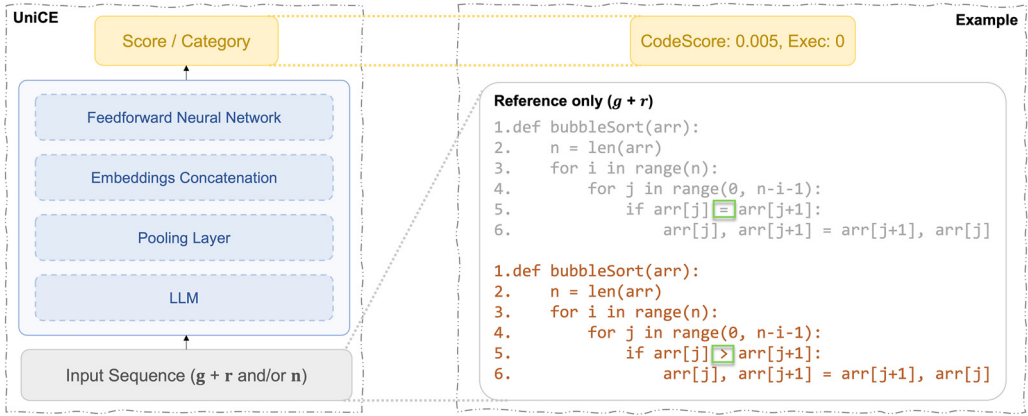


Fig. 3. Diagram of UniCE, where the left side of the figure shows its model architecture, and the right side of the figure shows the example (case in Figure 1) of input and output.

layer-wise attention mechanism and the final embedding of a token t can be computed as

$$e_t = \gamma \sum_{k=1}^l e_t^k h^k, \quad (8)$$

where l indicates the number of layers, and γ and h^k are trainable weights.

3.2.2 Unified Embedding. We require an efficient and comprehensive representation to encapsulate the unified input sequence x . Generally, there are two standard methods to extract the representation of x , i.e., averaging all token embeddings and using the first token embedding. While the first method is straightforward and includes information from all tokens, it may dilute the significance of more critical tokens and introduce extraneous noise. The first token of our base models is specifically designed to be a summary token⁶. Moreover, the work [42, 56] also proves the superiority of using the first token embedding compared to averaging all token embeddings in various applications. Thus, we employ the final embedding of the first token e_{first} as the representation of the unified input sequence x .

3.2.3 Unified Training. In UniCE, e_{first} is fed to a feed-forward neural network to output a score and/or a category. To unify three evaluation input formats into UniCE, we apply multi-task learning for training. Specifically, for each step, we assign three sub-steps for three input formats, yielding $\mathcal{L}^{Ref}$, $\mathcal{L}^{NL}$, and $\mathcal{L}^{Ref+NL}$, respectively. A Ref and NL data can be regarded as three input format data to yield three losses, while Ref-only and NL-only data can only compute the corresponding $\mathcal{L}^{Ref}$ and $\mathcal{L}^{NL}$. The final learning objective of UniCE is to minimize $\mathcal{L}^{Uni}$

$$\mathcal{L}^{Uni} = \mathcal{L}^{Ref} + \mathcal{L}^{NL} + \mathcal{L}^{Ref+NL}, \quad (9)$$

where $\mathcal{L}^{Ref}$, $\mathcal{L}^{NL}$, and $\mathcal{L}^{Ref+NL}$ are computed via Equation (4) using corresponding format data as input.

⁶During the pre-training of our base models (such as CodeBert, GraphCodeBert, and UniXcoder), the first input token is typically the CLS token (short for “classifier”), which enables the model to consider global contextual information during the encoding process through self-supervised learning methods. Therefore, the representation of this first token is usually used to represent the entire input sequence.

4 Evaluation

We aim at answering the following **research questions (RQs)**:

- RQ1: What is the performance of CodeScore on code evaluation tasks, compared to other EMs?
- RQ2: Can Exec effectively identify whether a generated code can be executed when all dependencies are met?
- RQ3: What is the contribution of L^{Uni} to UniCE for three input formats, compared to their respective losses?
- RQ4: How reasonable are the evaluations of CodeScore and other EMs from a human perspective?
- RQ5: How do CodeScore and other EMs perform on code evaluation tasks in a practical scenario?

Our five RQs aim to evaluate the efficacy and practicality of our approach compared to existing EMs. RQ1 and RQ4 assess our approach against current EMs through experiments and human evaluations, ensuring a comprehensive analysis from both quantitative and qualitative perspectives. RQ2 and RQ3 involve ablation studies to pinpoint the individual and combined impacts of our approach's main components. RQ5 evaluates our approach's real-world applicability through case studies.

4.1 Experiment Setup

In this section, we introduce datasets, baselines, correlation evaluation, and implementation details.

4.1.1 Datasets. We construct three public datasets (named APPS-Eval, MBPP-Eval, and HE-Eval) for code evaluation based on three public benchmark datasets in code generation, i.e., MBPP [2], APPS [18], and HumanEval [5].

To construct each code evaluation dataset, we first follow primitive NL and reference code in each corresponding base dataset. Then, for each paired NL and reference code in a code evaluation dataset, we generate an average of 20+ codes (generated from various LLMs, including CodeGen 350M and 16B [36], InCoder 1B and 6B [14], and CodeX 13B and 175B [5]. For HE-Eval dataset, we also consider the latest state-of-the-art LLMs including StarCoder 15.5B [28], CodeLlama 34B [49], and GPT-4 [38] besides the aforementioned LLMs.) according to NL and additionally build an average of 100+ correct test cases according to reference code. To obtain these test cases, the following steps were implemented:

- (1) Infer the type of input from pre-existing test cases.
- (2) Enumerate a collection of inputs constrained by the type of input and task.
- (3) Feed the input into the original correct code and get the output by execution (We assume that all external dependencies including third-party libraries have been installed correctly).

Finally, we label each matched NL, reference code, and generated code by executing the generated code with all corresponding test cases to compute PassRatio via Equation (1). Statistics of the datasets are presented in Tables 1 and 2.⁷ As demonstrated in Tables 1 and 2, there are notable disparities in the distributions of NL, RefCode (Reference Code), GenCode (Generated Code), and test cases across the three datasets. Specifically,

⁷For each generated code, we employ extended test cases of the corresponding task to compute its PassRatio and Passability. We compute the average number of PassRatio and Passability, i.e., AvgPassRatio and Pass@1, on the train, dev, and test sets of each dataset and display them in Table 2.

Table 1. Statistics of Datasets (Part 1)

Dataset	Examples Num			Avg Num/Task					Avg Length		
	Train	Dev	Test	NL	RefCode	GenCode	Extended (Original)	TestCase	NL	RefCode	GenCode
APPS-Eval	267,162	33,395	33,395	1	13	32	181 (13)		263.8	86.3	76.8
MBPP-Eval	15,679	3,000	3,000	1	1	24	102 (3)		15.5	32.5	26.7
HE-Eval	-	-	4221	1	1	26	108 (8)		61.9	24.4	41.6

Table 2. Statistics of Datasets (Part 2)

Dataset	AvgPassRatio			Pass@1		
	Train	Dev	Test	Train	Dev	Test
APPS-Eval	0.3196	0.1814	0.1790	0.0315	0.0007	0.0011
MBPP-Eval	0.2832	0.2571	0.2890	0.0674	0.0494	0.0760
HE-Eval	-	-	0.3695	-	-	0.1591

- *APPS-Eval* has 267,162 training examples and 33,395 examples each for dev and test sets. Each task typically includes 1 NL, 13 RefCode, and 42 GenCode, with average token lengths of 263.8 for NL, 86.3 for RefCode, and 76.8 for GenCode. Extended test cases average 181 per task, compared to the original 13. The AvgPassRatio for train, dev, and test sets are 0.3196, 0.1814, and 0.1790, respectively, while Pass@1 are 0.0315, 0.0007, and 0.0011, respectively.
- *MBPP-Eval* has 15,679 training examples and 3,000 examples each for dev and test sets. Each task typically includes 1 NL, 1 RefCode, and 24 GenCode, with average token lengths of 15.5 for NL, 32.5 for RefCode, and 26.7 for GenCode. Extended test cases average 102 per task, compared to the original 3. The AvgPassRatio for train, dev, and test sets are 0.2832, 0.2571, and 0.2890, respectively, while Pass@1 are 0.0674, 0.0494, and 0.0760, respectively.
- *HE-Eval* has 4,221 test examples. Each task typically includes 1 NL, 1 RefCode, and 26 GenCode, with average token lengths of 61.9 for NL, 24.4 for RefCode, and 41.6 for GenCode. Extended test cases average 108 per task, compared to the original 8. The AvgPassRatio for the test set is 0.3695, while Pass@1 is 0.1591.

4.1.2 Baselines. We select typical match-based CEMs, LLM-based EMs, and execution-based CEMs as baselines. We present each type of EMs as below.

Match-based CEMs include BLEU [39], Exact Matching Accuracy (Accuracy), CodeBLEU [48], and CrystalBLEU [12], specifically:

- *BLEU* [39] is calculated based on n-gram, and the fluency and correctness of generated code are expressed by calculating the proportion of n consecutive tokens in the correct code, where n is usually set to 4 (i.e., BLEU-4). Considering that shorter codes usually have higher BLEU values, a penalty item is introduced to BLEU as

$$BLEU = BP \cdot \exp \left(\sum_{m=1}^n \omega_m \log p_m \right),$$

$$BP = \begin{cases} 1, & l_g \geq l_r \\ e^{\left\{1 - \frac{r}{l_g}\right\}}, & l_g < l_r \end{cases},$$

where BP represents the penalty item, l_g represents the length of generated code, l_r represents the length of reference code, and ω_m and p_m represents the weighted coefficient and precision of m -gram, respectively.

- *Accuracy* indicates the percentage of exact matches between generated code and reference code.
- *CodeBLEU* [48] additionally takes into account the structure of code, which absorbs the advantages of BLEU in n -gram matching, and further injects code syntax through abstract syntax tree and code semantics through data flow

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} + \delta \cdot \text{Match}_{\text{ast}} + \zeta \cdot \text{Match}_{\text{df}},$$

where α, β, δ , and ζ are weights (usually set to 0.25, as well as in this article), $\text{BLEU}_{\text{weight}}$ is a weighted BLEU with different weights for various tokens, $\text{Match}_{\text{ast}}$ is syntactic AST matching, which explores the syntactic information of the code, and Match_{df} is semantic dataflow matching, which considers the semantic similarity between generated code and reference code.

- *CrystalBLEU* [12] is a metric that calculates BLEU by reducing the noise caused by trivially shared n -grams, such as “(” and “.”.

LLM-based EMs contain two well-known and widely used text EMs (BERTScore [63] and COMET [46]) and a concurrent work (CodeBERTScore [67]), specifically:

- *BERTScore* [63] is an automatic EM for text generation, which computes a similarity score for each token in the generated sentence with each token in the reference sentence with contextual embeddings of BERT [6]

$$R_{\text{BERT}} = \frac{1}{|\mathbf{x}|} \sum_{\mathbf{x}_i \in \mathbf{x}} \max_{\hat{\mathbf{x}}_j \in \hat{\mathbf{x}}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{\mathbf{x}}|} \sum_{\hat{\mathbf{x}}_j \in \hat{\mathbf{x}}} \max_{\mathbf{x}_i \in \mathbf{x}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \\ F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}.$$

Following the setting in [63], we compute BERTScore with inverse document frequency computed from test sets.

- *COMET* [46] provides a text EM by learning human judgments of training data, which leverages cross-lingual pre-trained language modeling to predict the quality of generated text more accurately.
- *CodeBERTScore* [67] is a concurrent work that tries to use the same way as BERTScore with LLM pre-trained on code.

Execution-based CEM refers to AvgPassRatio [18].

- *AvgPassRatio* [18] is defined as the average proportion of test cases that generated codes $\mathbf{g}_p$'s pass

$$\text{AvgPassRatio} = \frac{1}{|P|} \sum_{p \in P} \frac{1}{|C_p|} \sum_{c \in C_p} \mathbb{I} \{ \text{Eval}(\mathbf{g}_p, \mathcal{I}_{p,c}) = O_{p,c} \}, \quad (10)$$

where $|\cdot|$ indicates the element number of a set, $\mathbb{I}(\cdot)$ is an indicator function, which outputs 1 if the condition is true and 0 otherwise, and $\text{Eval}(\mathbf{g}_p, \mathcal{I}_{p,c})$ represents an evaluation function that obtains outputs of code $\mathbf{g}_p$ by way of executing it with $\mathcal{I}_{p,c}$ as input.

As mentioned above, continuous PassRatio (the item of AvgPassRatio) can better reflect the execution similarity of different codes than binary Passability (the item of Pass@1⁸). Therefore, in this article, we mainly compare the correlation between CodeScore and AvgPassRatio in Execution-based CEMs.

The input format of the proceeding baselines is Ref-only and each of them except COMET is in the range of 0 to 1.

4.1.3 Correlation Evaluation. We use three major correlation coefficients in statistics (i.e., Kendall-Tau (τ), Spearman R (r_s), and Pearson R (r_p)) to evaluate the correlation between each EM and functional correctness. Furthermore, we use **mean absolute error (MAE)** to assess the absolute error between them.

- *Kendall-Tau* (τ) [23] is a statistic used to measure the ordinal association between two measured data

$$\tau = \frac{\text{Concordant} - \text{Discordant}}{\text{Concordant} + \text{Discordant}}, \quad (11)$$

where *Concordant* indicates the number of occurrences that two evaluation data M^1 and M^2 exist either both $M_i^1 > M_j^1$ and $M_i^2 > M_j^2$ or both $M_i^1 < M_j^1$ and $M_i^2 < M_j^2$, and *Discordant* indicates the number of occurrences opposite to *Concordant*.

- *Spearman R* (r_s) [34] is a nonparametric measure of rank correlation (statistical dependence between the rankings of two data)

$$r_s = \frac{\text{cov}(R(M^1), R(M^2))}{\sigma_{R(M^1)} \sigma_{R(M^2)}}, \quad (12)$$

where $R(M^1)$ and $R(M^2)$ represent the rankings of M^1 and M^2 , $\text{cov}(\cdot, \cdot)$ means the covariance function, and σ_M means the standard deviation of M .

- *Pearson R* (r_p) [4] is a measure of linear correlation between two data

$$r_s = \frac{\text{cov}(M^1, M^2)}{\sigma_{M^1} \sigma_{M^2}}. \quad (13)$$

- *MAE* is a measure of errors between paired data

$$\text{MAE} = \frac{\sum_{i=1}^N |M_i^1 - M_i^2|}{N}, \quad (14)$$

where $|\cdot|$ means the absolute-value function.

4.1.4 Implementation Details. In this article, UniXcoder [15] is employed as the base LLM of UniCE, which has the similar parameter size of LLMs in BERTScore [63] and COMET [46], and larger LLMs can usually lead to better results. We format the input sequences as “[CLS] g [SEP] r [SEP] n [SEP],” where [CLS] and [SEP] are the special tokens in vocabulary, and we replace g , r , and n with the generated code, reference code, and NL description, respectively. For the balance of three input formats during the training process, we first sample an NL along with its corresponding generated code and reference code. They are then employed to construct data in three formats: Ref-only, NL-only, and Ref and NL. Finally, these formats are combined for training UniCE. In all experiments of this article, we train UniCE on the train set of APPS-Eval. We fine-tune UniCE on the train set of MBPP-Eval only when we specially mention it in our article. We train UniCE with

⁸ Pass@1 [25] is defined as the percentage of g_p s that pass all test cases of the corresponding p : $\frac{1}{|P|} \sum_{p \in P} \frac{1}{|C_p|} \prod_{c \in C_p} \mathbb{I}\{\text{Eval}(g_p, I_{p,c}) = O_{p,c}\}$, where Pass@1 is a more stringent CEM, also known as Strict Accuracy.

Table 3. Correlation Comparison of Functional Correctness on APPS-Eval Dataset

Method	Value	$\tau \uparrow$	$r_s \uparrow$	$r_p \uparrow$	MAE $\downarrow$	Execution Time $\downarrow$
Match-based CEM						
BLEU [39]	0.0094	0.1055	0.1156	0.0959	0.1164	$1.0 \times (26.0 \text{ s})$
Accuracy	0.0001	0.0079	0.0095	0.0196	-	$0.1 \times$
CodeBLEU [48]	0.2337	0.1035	0.1533	0.1085	0.2005	$7.8 \times$
CrystalBLEU [12]	0.0242	0.0906	0.1347	0.0887	0.1709	$0.3 \times$
LLM-based EM						
BERTScore [63]	0.8629	0.0916	0.1375	0.0718	0.6874	$56.7 \times$
COMET [46]	0.0165	0.0904	0.1126	0.1187	0.1751	$84.0 \times$
CodeBERTScore [67]	0.7583	0.1219	0.1801	0.1323	0.5885	$27.8 \times$
CodeScore						
Ref-only (g + r)						
UniCE with $\mathcal{L}^{Ref}$	0.1996	0.4760	0.6473	0.6620	0.1202	$33.7 \times$
UniCE with $\mathcal{L}^{Uni}$	0.1977	0.5033	0.6693	0.6929	0.1128	

NL-only (g + n)						
UniCE with $\mathcal{L}^{NL}$	0.2035	0.4679	0.6359	0.6855	0.1189	$37.9 \times$
UniCE with $\mathcal{L}^{Uni}$	0.2016	0.4901	0.6486	0.6905	0.1120	

Ref and NL (g + r + n)						
UniCE with $\mathcal{L}^{Ref+NL}$	0.1837	0.3865	0.5419	0.6152	0.1274	$44.2 \times$
UniCE with $\mathcal{L}^{Uni}$	0.1820	0.5275 ($\uparrow 40.56\%$)	0.7040 ($\uparrow 55.07\%$)	0.7210 ($\uparrow 58.87\%$)	0.1044	
Execution-based CEM						
13 test cases per task	0.0978	0.3360	0.4108	0.4987	0.1327	$1.5\text{k} \times$
181 test cases per task	0.1790	-	-	-	-	$20.7\text{k} \times$

The bold values denote the best performance. The red values indicate the improvements in comparison to the best performance of baselines.

Adam [24] optimizer on a single GPU of Tesla A100-PCIe-40G. Empirically, the learning rate is set to 0.001 and the training epoch is set to 5. The feedforward neural network of UniCE consists of 3 linear transitions with the hyperbolic tangent (Tanh) activation functions, where the corresponding output dimensions are 3,072, 1,024, and 2, respectively. The input token length is limited to 1,024. To mitigate the instability of model training, we exhibit the average performance of UniCE running five times.

4.2 Experimental Results

4.2.1 RQ1: Effect of CodeScore. As illustrated in Table 3, CodeScore exhibits a significantly stronger correlation with functional correctness than existing match-based CEMs and LLM-based EMs, which display weak or extremely weak correlations with Ground Truth on APPS-Eval. Compared with the top-performing EM among other EMs, CodeScore achieved absolute improvements of 40.56%, 55.07%, and 58.87% on τ , r_s , and r_p , respectively. With an r_s value greater than 0.6, it is evident that there is a strong correlation between CodeScore and Ground Truth. Furthermore, CodeScore has the lowest MAE compared to other EMs. The execution time of CodeScore is similar to other LLM-based EMs and slightly longer than existing Match-based CEMs. However, compared to the $20.7\text{k} \times$, $28.7\text{k} \times$, and $22.1\text{k} \times$ execution time of execution-based CEMs in three code evaluation datasets, CodeScore reduces execution time by three orders of magnitude. We also find that computing execution-based CEMs for code evaluation with a small number of original test cases is insufficient. They have a significant reduction in correlation coefficients compared to using larger extended test cases. In cases where test cases are rare or low-quality, such as on APPS-Eval, the correlation between our CodeScore and Ground Truth even far exceeds that of execution-based CEMs.

Table 4. Correlation Comparison of Functional Correctness on MBPP-Eval and HE-Eval Datasets

Method	MBPP-Eval			HE-Eval		
	Value	$r_s \uparrow$	Execution Time $\downarrow$	Value	$r_s \uparrow$	Execution Time $\downarrow$
Match-based CEM						
BLEU [39]	0.1186	0.1784	$1.0 \times (0.87 \text{ s})$	0.2436	0.0987	$1.0 \times (1.96 \text{ s})$
Accuracy	0.0004	0.0299	$0.1 \times$	0.0011	0.0456	$0.1 \times$
CodeBLEU [13]	0.1827	0.2902	$5.0 \times$	0.3452	0.3308	$6.3 \times$
CrystalBLEU [12]	0.0295	0.1645	$0.3 \times$	0.0427	0.2171	$0.4 \times$
LLM-based EM						
BERTScore [63]	0.8842	0.1522	$62.0 \times$	0.9008	0.1214	$57.5 \times$
COMET [46]	-0.5001	0.2681	$69.0 \times$	0.0879	0.1437	$58.2 \times$
CodeBERTScore [67]	0.7863	0.2490	$44.9 \times$	0.8091	0.3196	$47.4 \times$
CodeScore						
Ref-only ($g + r$)						
UniCE with $\mathcal{L}^{Ref}$	0.2975	0.5864	$17.2 \times$	0.3426	0.5671	$30.2 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3253	0.5999		0.4257	0.6378	
NL-only ($g + n$)						
UniCE with $\mathcal{L}^{NL}$	0.3364	0.4492	$12.6 \times$	0.4985	0.5634	$30.6 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3327	0.5719		0.5624	0.6215	
Ref and NL ($g + r + n$)						
UniCE with $\mathcal{L}^{Ref+NL}$	0.2905	0.5926	$20.7 \times$	0.4059	0.5965	$32.9 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3247	0.6027 ($\uparrow 31.25\%$)		0.4731	0.6597 ($\uparrow 32.89\%$)	
Execution-based CEM						
8 test cases per task	0.2670	0.6826	$1.0k \times$	0.5994	0.6981	$1.9k \times$
108 test cases per task	0.2890	-	$28.7k \times$	0.3695	-	$21.7k \times$

The bold values denote the best performance. The red values indicate the improvements in comparison to the best performance of baselines.

Table 5. Correlation Comparison of Functional Correctness with Different Base Models on HE-Eval Dataset

Method	Value	$\tau \uparrow$	$r_s \uparrow$	$r_p \uparrow$	MAE $\downarrow$	Execution Time $\downarrow$
CodeScore (UniCE with $\mathcal{L}^{Uni}$)						
UniXcoder	0.4731	0.4997	0.6597	0.6486	0.2179	$1.00 \times$
CodeBert	0.4809	0.4675	0.6236	0.5622	0.2344	$0.98 \times$
CodeGraphBert	0.4597	0.5073	0.6728	0.6480	0.2281	$1.07 \times$

We also sought to determine the generalizability. In Table 4, we utilize CodeScore, trained on APPS-Eval, to evaluate the code in MBPP-Eval and HE-Eval with fine-tuning and zero-shot settings, respectively. It is important to note that the distributions of NL, RefCode, GenCode, and test cases across these three datasets are quite different,⁹ as evidenced by their respective statistics shown in Tables 1 and 2. Table 4 reveals the effectiveness of CodeScore on MBPP-Eval and HE-Eval. Remarkably, CodeScore continues to achieve the best correlation compared to other match-based CEMs and LLM-based EMs in these two settings.

We conduct the experiments of UniCE based on different code pre-trained models, including CodeBert, CodeGraphBert, and UniXcoder. The results of the experiments are presented in Table 5.

⁹The average length of NL, RefCode, and GenCode across these three datasets are quite different. The average length of NL in APPS-Eval is 263.8, which far exceeds MBPP-Eval (15.5) and HE-Eval (61.9). The trend of the average length of RefCode and GenCode is similar to NL. For the Average Number of test cases per task, APPS-Eval is extended from 13 to 181, while MBPP-Eval and HE-Eval are extended from 3 and 8 to 102 and 108, respectively.

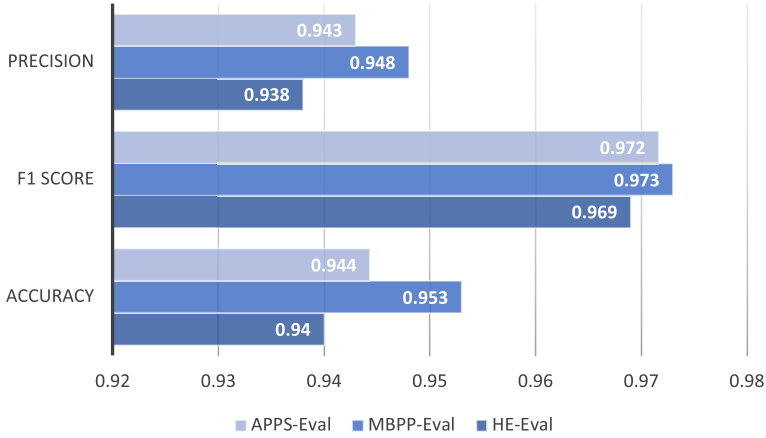


Fig. 4. The performance of Exec on APPS-Eval, MBPP-Eval, and HE-Eval datasets.

We did not observe obvious biases when choosing different base models. One trend we observed is that the better the model's ability to understand the code, the more accurate it is in evaluating the code.

Another intriguing finding is that the quality of CodeBLEU inversely correlates with code length. In other words, the longer code, the poorer correlation between CodeBLEU and Ground Truth. This is likely due to the fact that longer codes tend to incorporate more variations in their syntactic structure. Therefore, for longer codes, the evaluation effect of CodeBLEU gradually degrades to BLEU.

Summary of RQ1: CodeScore outperforms match-based CEMs and LLM-based EMs in terms of correlation with functional correctness, even on datasets that it was not trained on. Moreover, CodeScore operates at a speed three orders of magnitude faster than execution-based CEMs.

4.2.2 RQ2: Effect of Exec. We also evaluate the performance of Exec on APPS-Eval, MBPP-Eval, and HE-Eval datasets, as shown in Figure 4. The experimental results indicate that Exec demonstrates remarkably high performance in terms of Precision, F1 Score, and Accuracy. Through a comprehensive analysis of all datasets, we find that our approach's performance on the APPS-Eval dataset is inferior to that on the MBPP-Eval dataset. This discrepancy is primarily due to the higher complexity and length of problems in the APPS-Eval dataset compared to those in MBPP-Eval. Furthermore, the performance on the HE-Eval dataset is the poorest, because our approach has not been trained on this dataset. Nevertheless, our approach's performance across various metrics on the HE-Eval dataset exceeded 90% in the zero-shot setting, indicating its effective transferability to unseen datasets. These results prove that using UniCE to learn code execution is effective for code evaluation.

Summary of RQ2: The Exec component in our approach demonstrates extremely high Precision/F1 Score/Accuracy in determining whether the code can be executed when all dependencies are met.

4.2.3 RQ3: Effect of $\mathcal{L}^{Uni}$. As observed from Tables 3 and 4, our proposed $\mathcal{L}^{Uni}$ demonstrates enhancements across all input formats when compared to their respective losses on APPS-Eval, MBPP-Eval, and HE-Eval datasets. With changes in the input format, both the correlation coefficients

Table 6. Human Evaluation for Functional Correctness

EM	Reasonableness of Evaluation
BERTScore [63]	1.3 ± 0.4
CodeBLEU [48]	2.1 ± 0.5
CodeBERTScore [67]	2.2 ± 0.7
CodeScore	$3.4 (\uparrow 54.6\%) \pm 0.3$
Ground Truth	4.6 ± 0.2

The bold values denote the best performance. The red values indicate the improvements in comparison to the best performance of baselines.

and MAE between CodeScore and Ground Truth also vary. Generally, the Ref and NL input format yields superior results, which shows that accommodating NL has a positive effect on evaluating the generated code, while the traditional Ref-only input format omits the valuable information in NL. Additionally, according to the Avg Length data presented in Table 1, we discovered that the execution time of CodeScore exhibits a linear, positive relationship with the input length. Regardless of the input formats, our proposed CodeScore provides a commendable evaluation of generated code. This is attributable to the fact that $\mathcal{L}^{Uni}$ aids in training a code evaluation model with a unified input.

Summary of RQ3: The component $\mathcal{L}^{Uni}$ in our approach shows positive effects across different input formats.

4.2.4 RQ4: Human Evaluation. In this section, we conduct a human evaluation to gauge the validity of our CodeScore. Considering the costliness of human evaluation, we select only five representative EMs for this task, namely, CodeScore, CodeBLEU, BERTScore, CodeBERTScore, and Ground Truth (i.e., PassRatio). All of these EMs are continuous and range from 0 to 1. In accordance with previous work [17] and our experimental setup, we manually assess the validity of each EM in gauging the functional correctness of the generated code. The score for this evaluation is an integer ranging from 0 to 5, where 0 denotes poor and 5 signifies excellent performance.

The human evaluation is conducted on the Python dataset HE-Eval. We randomly select 100 samples¹⁰ from this dataset, each consisting of NL descriptions, reference code, and generated code. These samples are scored using five EMs, resulting in a total of 100×5 data pairs. We invite ten computer science PhD students, each with over three years of experience in Python development, to serve as evaluators. The 500 code snippets are divided into 10 groups, with each questionnaire containing one group. We randomly list the generated code with reference code and NL and the corresponding EM score on the questionnaire. Each group is evaluated anonymously by one evaluator, and the final score is the average of all evaluators' scores. Evaluators are allowed to search the Internet for unfamiliar concepts.

We present the results of the human evaluation in Table 6. Remarkably, our proposed CodeScore significantly outperforms all other EMs. Relative to these, CodeScore shows an improvement of at least 54.6% in the human evaluation. All p-values are substantially less than 0.005,¹¹ underscoring that these improvements are statistically significant.

¹⁰Considering the workload of the evaluators, we choose a moderate sample size of 100. Too many samples would exceed the evaluators' capacity.

¹¹The smaller the p-value, the less likely it is that the results are due to random factors.

NL	Write a function to find numbers divisible by m and n from a list of numbers using lambda function.	
Reference Code	<pre>def div_of_nums(nums,m,n): result = list(filter(lambda x: (x % m == 0 and x % n == 0), nums)) return result</pre>	
Generated Code I.1 ❌		Generated Code I.2 ✅
<pre>def div_of_nums(nums,m,n): ans = [] for n in nums: if n%m == 0: ans.append(n) return ans</pre>		<pre>def div_of_nums(nums,m,n): return [x for x in nums if x % m == 0 and x % n == 0]</pre>
CodeScore: 0.4159 CodeScore (NL-only): 0.4946 CodeScore (Ref-only): 0.3982 CodeBLEU: 0.2283 BERTScore: 0.8862 CodeBERTScore: 0.8615		CodeScore: 0.9971 CodeScore (NL-only): 0.8323 CodeScore (Ref-only): 0.9631 CodeBLEU: 0.2456 BERTScore: 0.9005 CodeBERTScore: 0.8484

(a) Case I

NL	Write a python function to check whether the given two numbers have same number of digits or not.	
Reference Code	<pre>def same_Length(A,B): while (A > 0 and B > 0): A = A / 10; B = B / 10; if (A == 0 and B == 0): return True; return False;</pre>	
Generated Code II.1 ❌		Generated Code II.2 ❌
<pre>def same_Length(iterable, n): p = (n-1) * iterable return p</pre>		<pre>def same_Length(A,B): if len(str(A))==len(str(B)): return True else: try: float(str(str(B))) except ValueError: return False else: return False</pre>
CodeScore: 0.1272 CodeScore (NL-only): 0.0962 CodeScore (Ref-only): 0.1540 CodeBLEU: 0.1383 BERTScore: 0.8542 CodeBERTScore: 0.7021		CodeScore: 0.5712 CodeScore (NL-only): 0.6442 CodeScore (Ref-only): 0.5419 CodeBLEU: 0.1744 BERTScore: 0.8454 CodeBERTScore: 0.7816

(b) Case II

NL	Write a function to return true if the given number is even else return false.	
Reference Code	<pre>def even_num(x): if x%2==0: return True else: return False</pre>	
Generated Code III.1 ❌		Generated Code III.2 ✅
<pre>def even_num(n,count = 0): if (n == 0): return False if (count % 2 == 101): return (n % 2 == 3) else: return (2 * n - 1) + (n % 4 - 1) > 1</pre>		<pre>def even_num(x): return True if x % 2 == 0 else False</pre>
CodeScore: 0.3590 CodeScore (NL-only): 0.4350 CodeScore (Ref-only): 0.2988 CodeBLEU: 0.3582 BERTScore: 0.8882 CodeBERTScore: 0.7764		CodeScore: 0.9904 CodeScore (NL-only): 1.0000 CodeScore (Ref-only): 0.9710 CodeBLEU: 0.3421 BERTScore: 0.8665 CodeBERTScore: 0.8345

(c) Case III

Fig. 5. Case study of different EMs. For each case, the second generated code is superior to the first one.

Summary of RQ4: Human evaluation indicates that CodeScore shows significant improvements over previous representative EMs.

4.2.5 RQ5: Case Study. Figure 5 displays a selection of generated codes and their corresponding EM scores (as per Section 4.2.4) on MBPP-Eval dataset. It becomes evident that CodeBLEU, BERTScore, and CodeBERTScore each exhibit unique issues. From these examples, we glean the following insights: (1) CodeBLEU tends to assign relatively low scores to generated code, even when the code is functionally correct. Furthermore, it appears to favor generated codes that maintain

NL	You'll be given a string of words, and your task is to count the number of boredom. A boredom is a sentence that starts with the word "I". Sentences are delimited by '.', '?' or '!'. 	
Reference Code	<pre>def is_bored(S): import re sentences = re.split(r'[.?!]\s*', S) return sum(sentence[0:2] == 'I' for sentence in sentences)</pre>	
Generated Code 1 ❌		Generated Code 2 ✅
<pre>def is_bored(S): return ['I' in i for i in S.split(' ')]</pre> Exec: 0		<pre>def is_bored(S): return len(filter(lambda x: x[0] == 'I', S.split()))</pre> Exec: 1

Fig. 6. Case study of Exec.

structural consistency with the reference code. For instance, even though Generated Code III.2 is functionally correct, it receives a lower CodeBLEU score than III.1, which is fundamentally incorrect. (2) Both BERTScore and CodeBERTScore have a propensity to award relatively high scores to generated code, even when the code is essentially flawed. Additionally, they often assign lower scores to better generated codes. For example, Generated Code II/III.2 has a lower BERTScore than II/III.1, and Generated Code I.2 has a lower CodeBERTScore than I.1. In contrast, CodeScore performs admirably in all of these scenarios. Our CodeScore aligns more closely with Ground Truth compared to other EMs. Moreover, the various formats of input have little impact on CodeScore's scorings, indicating that CodeScore can effectively make judgments based on NL and/or reference code, adapting to different input formats.

We further examine Exec's capabilities through a case study. We find that Exec can effectively discriminate the cases of successful and unsuccessful compilation, especially sensitive to some errors that lead to compilation failures. A representative example is shown in Figure 6, where in Generated Code 1, the code with mismatched parentheses is recognized by Exec, and in Generated Code 2, the code with multiple nested parentheses is not misidentified by Exec.

Summary of RQ5: Through case studies, we find that our approach does not have the problems faced by previous EMs and is effective in evaluating the functional correctness and compilability of generated code.

5 Threats to Validity

There are two major threats to the validity of our work. (1) Threats to external validity concern the quality of experimental datasets and the generalizability of our results. We evaluated our approach using three public code generation datasets, which are considered mainstream benchmarks in the field and have been utilized extensively in prior research [19, 20, 28, 33, 59, 62]. Given their widespread use, we believe that the findings derived from these datasets offer a reasonable degree of generalizability and could potentially extend to other datasets. (2) Threats to internal validity involve the impact of hyperparameters and instability characteristics of deep learning models. Deep learning models exhibit a certain sensitivity to hyperparameter settings. In our approach, we conduct a small-range grid search on hyper-parameters using a distinct validation subset. The same set of hyperparameters is consistently applied across all datasets and compared with various baselines, achieving favorable performance consistently. Even with the same hyper-parameters, deep learning models still encounter instability issues due to factors such as the random initialization of model parameters and the random shuffling of training data. Therefore, in our experiments, we run UniCE 5 times and report its average performance. For fairness, we also run other LLM-based metrics five times with their public source code and provide the average performance.

6 Discussion

While we have demonstrated that CodeScore is an effective LLM-based metric for code evaluation, we acknowledge that it still has certain limitations.

- First, learning code execution for code evaluation requires collecting a certain amount of data, including sufficient test cases, generated codes, reference codes, and NL descriptions. However, collecting this data is far less expensive than performing human evaluation.
- Second, in this article, CodeScore is more suitable for evaluating function-level code in Python. Nevertheless, our work establishes the viability of code evaluation based on UniCE, and this approach can feasibly be extended to other scenarios. We aim to broaden CodeScore to encompass a wider range of codes in our future work.
- Third, employing CodeScore for code evaluation entails additional computation and time. However, we maintain that this is still within an acceptable range, considering the benefits it provides in terms of the accuracy and reliability of code evaluation.

7 Conclusion and Future Work

In this article, we have proposed a code evaluation learning framework based on LLMs with a unified input, which we refer to as UniCE. UniCe is designed to learn the code execution of generated code. In response to the imprecise evaluations provided by existing match-based CEMs and LLM-based EMs, we introduced CodeScore based on UniCE, which is an effective CEM to measure the functional correctness of generated code. Furthermore, our CodeScore can be applied to three application scenarios (Ref-only, NL-only, and Ref and NL) for code evaluation with a unified input. This is in contrast to traditional CEMs, which typically only consider the Ref-only scenario. To validate CodeScore, we constructed three code evaluation datasets (i.e., APPS-Eval, MBPP-Eval, and HE-Eval), which correspond to three popular benchmark datasets in code generation (i.e., MBPP, APPS, and HumanEval). Experimental results affirm the efficacy of CodeScore, which achieves state-of-the-art performance on multiple code evaluation datasets.

We hope this work sheds light on future work in the direction of LLM-based code evaluation. Our code evaluation dataset can serve as a benchmark for evaluating the functional correctness of generated code. Furthermore, our work can be applied to facilitate the training of code generation models by providing positive feedback.

References

- [1] Shushan Arakelyan, Anna Hakhverdyan, Miltiadis Allamanis, Christophe Hauser, Luis Garcia, and Xiang Ren. 2022. NS3: Neuro-symbolic semantic code search. arXiv:2205.10674. Retrieved from <https://arxiv.org/abs/2205.10674>
- [2] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program synthesis with large language models. arXiv:2108.07732. Retrieved from <https://arxiv.org/abs/2108.07732>
- [3] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *IEEValuation@ACL*. Association for Computational Linguistics, 65–72.
- [4] Auguste Bravais. 1844. *Analyse mathématique sur les probabilités des erreurs de situation d'un point*. Impr. Royale.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>

- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT (1)*. Association for Computational Linguistics, 4171–4186.
- [7] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7, Article 189 (2023), 38 pages. DOI: <https://doi.org/10.1145/3672459>
- [8] Yihong Dong, Xue Jiang, Huanyu Liu, Zhi Jin, and Ge Li. 2024. Generalization or memorization: Data contamination and trustworthy evaluation for large language models. arXiv:2402.15938. Retrieved from <https://arxiv.org/abs/2402.15938>
- [9] Yihong Dong, Ge Li, Xue Jiang, and Zhi Jin. 2023. Antecedent predictions are more important than you think: An effective method for tree-based code generation. In *ECAI '23*. IOS Press, 565–574.
- [10] Yihong Dong, Ge Li, and Zhi Jin. 2023. CODEP: Grammatical seq2seq model for general-purpose code generation. In *ISSTA*. ACM, 188–198.
- [11] Yihong Dong, Kangcheng Luo, Xue Jiang, Zhi Jin, and Ge Li. 2024. PACE: Improving prompt with actor-critic editing for large language model. In *ACL '24*. 7304–7323.
- [12] Aryaz Eghbali and Michael Pradel. 2022. CrystalBLEU: Precisely and efficiently measuring the similarity of code. In *ASE*. ACM, 28:1–28:12.
- [13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *EMNLP (Findings)*. Association for Computational Linguistics, 1536–1547.
- [14] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A generative model for code infilling and synthesis. arXiv:2204.05999. Retrieved from <https://arxiv.org/abs/2204.05999>
- [15] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *ACL (1)*. Association for Computational Linguistics, 7212–7225.
- [16] Daya Guo, Alexey Svyatkovskiy, Jian Yin, Nan Duan, Marc Brockschmidt, and Miltiadis Allamanis. 2022. Learning to complete code with sketches. In *ICLR*. OpenReview.net.
- [17] Yiyang Hao, Ge Li, Yongqiang Liu, Xiaowei Miao, He Zong, Siyuan Jiang, Yang Liu, and He Wei. 2022. AixBench: A code generation benchmark dataset. arXiv:2206.13179. Retrieved from <https://arxiv.org/abs/2206.13179>
- [18] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks*.
- [19] Dong Huang, Qingwen Bu, Jie Zhang, Xiaofei Xie, Junjie Chen, and Heming Cui. 2023. Bias assessment and mitigation in llm-based code generation. arXiv:2309.14345. Retrieved from <https://arxiv.org/abs/2309.14345>
- [20] Jeevana Priya Inala, Chenglong Wang, Mei Yang, Andrés Coda, Mark Encarnación, Shuvendu K. Lahiri, Madanlal Musuvathi, and Jianfeng Gao. 2022. Fault-aware neural code rankers. In *NeurIPS*.
- [21] Xue Jiang, Yihong Dong, Zhi Jin, and Ge Li. 2024. SEED: Customize large language models with sample-efficient adaptation for code generation. arXiv:2403.00046.
- [22] Xue Jiang, Yihong Dong, Lecheng Wang, Fang Zheng, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2023. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7, Article 182 (2023), 30 pages. DOI: <https://doi.org/10.1145/3672456>
- [23] Maurice G. Kendall. 1938. A new measure of rank correlation. *Biometrika* 30, 1/2 (1938), 81–93.
- [24] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *ICLR*.
- [25] Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy Liang. 2019. SPoC: Search-based pseudocode to code. In *NeurIPS*, 11883–11894.
- [26] Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. 2024a. EvoCodeBench: An evolving code generation benchmark aligned with real-world code repositories. arXiv:2404.00599. Retrieved from <https://arxiv.org/abs/2404.00599>
- [27] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, Yongbin Li, Bin Gu, and Mengfei Yang. 2024b. DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories. In *ACL '24*, 3603–3614.
- [28] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Moustafa-Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan

- Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from <https://arxiv.org/abs/2305.06161>
- [29] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [30] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, 74–81.
- [31] Wang Ling, Phil Blunsom, Edward Grefenstette, Karl Moritz Hermann, Tomás Kociský, Fumin Wang, and Andrew W. Senior. 2016. Latent predictor networks for code generation. In *ACL*, Vol. 1. The Association for Computer Linguistics.
- [32] Shuai Lu, Nan Duan, Hjoae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. ReACC: A retrieval-augmented code completion framework. In *ACL*. Association for Computational Linguistics, 6227–6240.
- [33] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering code large language models with evol-instruct. arXiv:2306.08568. Retrieved from <https://arxiv.org/abs/2306.08568>
- [34] Alexander McFarlane Mood. 1950. *Introduction to the Theory of Statistics*. McGraw-Hill
- [35] Rohan Mukherjee, Yeming Wen, Dipak Chaudhari, Thomas W. Reps, Swarat Chaudhuri, and Christopher M. Jermaine. 2021. Neural program generation modulo static analysis. In *NeurIPS*, 18984–18996.
- [36] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An open large language model for code with multi-turn program synthesis. In *ICLR*. OpenReview.net.
- [37] OpenAI. 2023. ChatGPT: Optimizing Language Models for Dialogue. Retrieved from <https://openai.com/blog/chatgpt/>
- [38] OpenAI. 2023. GPT-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [39] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *ACL*. ACL, 311–318.
- [40] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep contextualized word representations. In *NAACL-HLT*. Association for Computational Linguistics, 2227–2237.
- [41] Maja Popovic. 2015. chrF: Character n-gram F-score for automatic MT evaluation. In *WMT@EMNLP*. The Association for Computer Linguistics, 392–395.
- [42] Tharindu Ranasinghe, Constantin Orasan, and Ruslan Mitkov. 2020. TransQuest: Translation quality estimation with cross-lingual transformers. In *COLING*. International Committee on Computational Linguistics, 5070–5081.
- [43] Veselin Raychev, Martin T. Vechev, and Eran Yahav. 2014. Code completion with statistical language models. In *PLDI*. ACM, 419–428.
- [44] Ricardo Rei, José GC De Souza, Duarte Alves, Chrysoula Zerva, Ana C Farinha, Taisiya Glushkova, Alon Lavie, Luisa Coheur, and André FT Martins. 2022. COMET-22: Unbabel-IST 2022 submission for the metrics shared task. In *WMT@EMNLP*. Association for Computational Linguistics, 578–585.
- [45] Ricardo Rei, Ana C. Farinha, Chrysoula Zerva, Daan van Stigt, Craig Stewart, Pedro G. Ramos, Taisiya Glushkova, André F. T. Martins, and Alon Lavie. 2021. Are references really needed? Unbabel-IST 2021 submission for the metrics shared task. In *WMT@EMNLP*. Association for Computational Linguistics, 1030–1040.
- [46] Ricardo Rei, Craig Stewart, Ana C. Farinha, and Alon Lavie. 2020. COMET: A neural framework for MT evaluation. In *EMNLP*, Vol. 1. Association for Computational Linguistics, 2685–2702.
- [47] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *EMNLP/IJCNLP*, Vol. 1. Association for Computational Linguistics, 3980–3990.
- [48] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis. arXiv:2009.10297. Retrieved from <https://arxiv.org/abs/2009.10297>
- [49] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [50] Baptiste Rozière, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. In *NeurIPS*.
- [51] Thibault Sellam, Dipanjan Das, and Ankur P. Parikh. 2020. BLEURT: Learning robust metrics for text generation. In *ACL*. Association for Computational Linguistics, 7881–7892.

- [52] Sijie Shen, Xiang Zhu, Yihong Dong, Qizhi Guo, Yankun Zhen, and Ge Li. 2022. Incorporating domain knowledge through task augmentation for front-end JavaScript code generation. In *ESEC/SIGSOFT FSE*. ACM, 1533–1543.
- [53] Weisong Sun, Chunrong Fang, Yuchen Chen, Guan hong Tao, Tingxu Han, and Qunjun Zhang. 2022. Code search based on context-aware code translation. In *ICSE*. ACM, 388–400.
- [54] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. 2020. TreeGen: A tree-based transformer architecture for code generation. In *AAAI*. AAAI Press, 8984–8991.
- [55] Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. BERT rediscovers the classical NLP pipeline. In *ACL*, Vol. 1. Association for Computational Linguistics, 4593–4601.
- [56] Yu Wan, Dayiheng Liu, Baosong Yang, Haibo Zhang, Boxing Chen, Derek F. Wong, and Lidia S. Chao. 2022. UniTE: Unified translation evaluation. In *ACL*, Vol. 1. Association for Computational Linguistics, 8117–8127.
- [57] Yue Wang, Weishi Wang, Shafiq R. Joty, and teven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, Vol. 1. 8696–8708.
- [58] Bolin Wei, Ge Li, Xin Xia, Zhiyi Fu, and Zhi Jin. 2019. Code generation as a dual task of code summarization. In *NeurIPS*, 6559–6569.
- [59] Xiaokai Wei, Suran Kumar Gonugondla, Shiqi Wang, Wasi Uddin Ahmad, Baishakhi Ray, Haifeng Qian, Xiaopeng Li, Varun Kumar, Zijian Wang, Yuchen Tian, Qing Sun, Ben Athiwaratkun, Mingyue Shang, Murali Krishna Ramanathan, Parminder Bhatia, and Bing Xiang. 2023. Towards greener yet powerful code generation via quantization: An empirical study. In *ESEC/SIGSOFT FSE*. ACM, 224–236.
- [60] Pengcheng Yin and Graham Neubig. 2018. TRANX: A transition-based neural abstract syntax parser for semantic parsing and code generation. In *EMNLP*, 7–12.
- [61] Weizhe Yuan, Graham Neubig, and Pengfei Liu. 2021. BARTScore: Evaluating generated text as text generation. In *NeurIPS*, 27263–27277.
- [62] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with large language models for code generation. In *ICLR*. OpenReview.net.
- [63] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *ICLR*. OpenReview.net.
- [64] Wei Zhao, Maxime Peyrard, Fei Liu, Yang Gao, Christian M. Meyer, and Steffen Eger. 2019. MoverScore: Text generation evaluating with contextualized embeddings and earth mover distance. In *EMNLP/IJCNLP*, Vol. 1. Association for Computational Linguistics, 563–578.
- [65] Yunfei Zhao, Yihong Dong, and Ge Li. 2023. Seq2Seq or Seq2Tree: Generating code using both paradigms via mutual learning. In *Internetware*, 238–248.
- [66] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A pre-trained model for code generation with multilingual evaluations on Humaneval-X. arXiv:2303.17568. Retrieved from <https://arxiv.org/abs/2303.17568>
- [67] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. CodeBERTScore: Evaluating code generation with pretrained models of code. arXiv:2302.05527. Retrieved from <https://arxiv.org/abs/2302.05527>
- [68] Ming Zhu, Karthik Suresh, and Chandan K. Reddy. 2022. Multilingual code snippets training for program translation. In *AAAI*. AAAI Press, 11783–11790.

Appendix

A Details of Table 4

We provide the detailed information of Table 4 in Tables A1 and A2.

Table A1. Correlation Comparison of Functional Correctness on MBPP-Eval Datasets

Method	Value	$\tau \uparrow$	$r_s \uparrow$	$r_p \uparrow$	MAE $\downarrow$	Execution Time $\downarrow$
Match-based CEM						
BLEU	0.1186	0.1253	0.1784	0.1422	0.2827	$1.0 \times (0.87 \text{ s})$
Accuracy	0.0004	0.0257	0.0299	0.0345	-	$0.1 \times$
CodeBLEU	0.1827	0.2063	0.2902	0.2896	0.2738	$5.0 \times$
CrystalBLEU	0.0295	0.1645	0.2338	0.1021	0.2856	$0.3 \times$
LLM-based EM						
BERTScore	0.8842	0.1076	0.1522	0.1136	0.6271	$62.0 \times$
COMET	-0.5001	0.1900	0.2681	0.2167	0.8157	$69.0 \times$
CodeBERTScore	0.7863	0.1757	0.2490	0.2180	0.5596	$44.9 \times$
CodeScore						
Ref-only (g + r)						
UniCE with $\mathcal{L}^{Ref}$	0.2975	0.4362	0.5864	0.5896	0.2267	$17.2 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3253	0.4473 ($\uparrow 24.10\%$)	0.5999	0.5962	0.2253	

NL-only (g + n)						
UniCE with $\mathcal{L}^{NL}$	0.3364	0.3206	0.4492	0.4992	0.2640	$12.6 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3327	0.4178	0.5719	0.5837	0.2309	

Ref and NL (g + r + n)						
UniCE with $\mathcal{L}^{Ref+NL}$	0.2905	0.4377	0.5926	0.5997	0.2251	$20.7 \times$
UniCE with $\mathcal{L}^{Uni}$	0.3247	0.4466	0.6027 ($\uparrow 31.25\%$)	0.6054 ($\uparrow 31.58\%$)	0.2230	
Execution-based CEM						
8 test cases per task	0.2670	0.5921	0.6826	0.6478	0.1440	$1.0k \times$
108 test cases per task	0.2890	-	-	-	-	$28.7k \times$

The bold values denote the best performance. The red values indicate the improvements in comparison to the best performance of baselines.

Table A2. Correlation Comparison of Functional Correctness on HE-Eval Datasets

Method	Value	$\tau \uparrow$	$r_s \uparrow$	$r_p \uparrow$	MAE $\downarrow$	Execution Time $\downarrow$
Match-based CEM						
BLEU	0.2436	0.0669	0.0987	0.0998	0.3178	$1.0 \times (1.96 \text{ s})$
Accuracy	0.0011	0.0397	0.0456	0.0486	-	$0.1 \times$
CodeBLEU	0.3452	0.2369	0.3308	0.3147	0.2966	$6.3 \times$
CrystalBLEU	0.0427	0.1577	0.2171	0.2006	0.3398	$0.4 \times$
LLM-based EM						
BERTScore	0.9008	0.0893	0.1214	0.1155	0.5655	$57.5 \times$
COMET	0.0879	0.0916	0.1437	0.1279	0.6169	$58.2 \times$
CodeBERTScore	0.8091	0.2208	0.3196	0.3021	0.5012	$47.4 \times$
CodeScore						
Ref-only (g + r)						
UniCE with $\mathcal{L}^{Ref}$	0.3426	0.4305	0.5671	0.5575	0.2382	$30.2 \times$
UniCE with $\mathcal{L}^{Uni}$	0.4257	0.4842	0.6378	0.6269	0.2268	

NL-only (g + n)						
UniCE with $\mathcal{L}^{NL}$	0.4985	0.4277	0.5634	0.5639	0.2474	$30.6 \times$
UniCE with $\mathcal{L}^{Uni}$	0.5624	0.4718	0.6215	0.6270	0.2327	

Ref and NL (g + r + n)						
UniCE with $\mathcal{L}^{Ref+NL}$	0.4059	0.4529	0.5965	0.5864	0.2267	$32.9 \times$
UniCE with $\mathcal{L}^{Uni}$	0.4731	0.4959 ($\uparrow 25.90\%$)	0.6597 ($\uparrow 32.89\%$)	0.6486 ($\uparrow 33.39\%$)	0.2179	
Execution-based CEM						
8 test cases per task	0.5994	0.5682	0.6981	0.6656	0.1307	$1.9k \times$
108 test cases per task	0.3695	-	-	-	-	$21.7k \times$

The bold values denote the best performance. The red values indicate the improvements in comparison to the best performance of baselines.

Received 26 November 2023; revised 16 July 2024; accepted 19 August 2024